Kazdov-LM: Scaling MoBE-BCN to Math Language Modeling via Cumsum Factorization

Author: Juan Cruz Dovzak (independent researcher) **Contact:** juandovzak@gmail.com **Status:** In preparation — experiments ongoing **License:** CC BY-NC 4.0 (text) · MIT (reference implementation) · Patent rights retained **Code:** https://github.com/OriginalKazdov/kazdov

> ***This paper is a work-in-progress placeholder.*** *The algebraic-composition companion paper (Dovzak 2026a, `01-mobe-bcn.md`) contains the complete MoBE-BCN architecture and sample-efficiency results. This paper will be updated with final numbers once training and ablation runs complete.*
>
> ***Section status***
>
> - [x] *Sections 1-4 (architecture, cumsum derivation) — complete draft*
> - [ ] *Section 5 (training results) — pending ~24h alpha run*
> - [ ] *Section 6 (ablations) — pending D4 ablation sweep*
> - [ ] *Section 7 (qualitative samples) — pending training completion*
>
> *Last updated: April 2026.*

ABSTRACT (DRAFT)

MoBE-BCN (Dovzak 2026a) achieves state-of-the-art sample efficiency on algebraic composition tasks but a naive implementation materializes an $O(B \cdot L^2 \cdot D \cdot K)$ pairwise-composition tensor inside attention, infeasible at language-model scale. We derive a **cumsum-factorized attention kernel** that reduces MoBE-BCN attention to $O(B \cdot L \cdot r \cdot K)$ when the router depends only on the query side — aligning with standard token-level MoE routing and preserving expert specialization. Combined with a vectorized K-expert kernel, this yields a 259× throughput improvement on a single RTX 4090, enabling overnight training of a 98M-parameter math-specialized LM (Kazdov-α) on a 5.95B-token mathematics corpus. On held-out math loss, Kazdov-α `[PENDING]` a matched-tokenizer GPT-2 124M baseline. We release the model, code, and training recipe under MIT.

---

1. Introduction

Transformer attention is $O(L^2)$ in sequence length. Linear-attention variants (Katharopoulos et al., 2020; Choromanski et al., 2020) factor the softmax kernel into feature maps on Q and K that can be combined via cumulative sums, yielding $O(L)$. MoBE-BCN's bilinear composition $U((V^T x) \odot (W^T y))$ has precisely this structure — a tensor product that factorizes additively across the key axis — but the companion paper's naive implementation materializes the full pairwise tensor before reducing. This paper shows the factorization explicitly, derives the $O(L)$ attention kernel, and demonstrates it enables math-specialized language-model training on consumer hardware.

Contributions.

1. **Cumsum-factorized MoBE-BCN attention** (Section 3). A derivation and reference implementation that reduces memory from `O(B·L²·D·K)` to `O(B·L·r·K)` using query-side routing. Drop-in replacement for the naive `CausalMoBEBCNAttention`.
2. **Vectorized K-expert kernel** (Section 3.3). Stacked expert parameters with batched einsum; eliminates the Python per-expert loop.
3. **Kazdov-α training recipe** (Section 4). 98M-parameter math LM trained on a 5.95B-token mathematics corpus (OpenWebMath, proof-pile, MathPile, AutoMathText, NuminaMath) in ~24h on a single RTX 4090.
4. **Held-out math loss benchmark** (Section 5, *pending*). Comparison against a matched-tokenizer GPT-2 124M baseline on five held-out math-corpus slices.
5. **Ablation study** (Section 6, *pending*). K ∈ {2, 4, 8}, rank ∈ {64, 128, 256}, router type (Q-only vs Q+K), hybrid-MHA on/off.

---

2. Background

See Dovzak 2026a for the full MoBE-BCN architecture. The core primitive is

```
BCNₖ(x, y) = Uₖ · ((Vₖᵀ x) ⊙ (Wₖᵀ y)) + bₖ
MoBE(x, y) = Σ_{k=1..K} wₖ(x, y) · BCNₖ(x, y)
```

with `wₖ(x, y) = softmax(Router([x; y]))[k]`. In attention, this is applied as `outᵢ = Σ_{j ≤ i} MoBE(Qᵢ, Kⱼ) / n_valid(i)`.

2.1 Memory and compute complexity (naive)

The naive implementation expands `Q` and `K` to `(B, L, L, D)` via `unsqueeze`-and-`expand` before calling the composition:

```
Qᵢ_expanded = Q.unsqueeze(2).expand(B, L, L, D)   # O(B·L²·D)
Kⱼ_expanded = K.unsqueeze(1).expand(B, L, L, D)   # O(B·L²·D)
fwd = MoBE(Qᵢ_expanded, Kⱼ_expanded)              # O(B·L²·D·K)
```

For a target 98M-parameter model (D=512, K=4, 12 layers) at L=256, each layer must hold ~4 GB of intermediate activations. Combined with gradient storage and the `O(L²)` softmax-based parallel MHA channel, this exceeds 24 GB VRAM and is infeasible to train without gradient checkpointing penalties that halve throughput.

---

3. Cumsum-Factorized Attention

3.1 Key insight — bilinearity implies separability

The `j` index is the one we sum over in the causal attention. Because compose is *bilinear* in `Q` and `K`, the sum over `j` factors through the Hadamard product:

```
Σ_{j ≤ i} BCN(Qᵢ, Kⱼ)
  = Σⱼ U · ((Vᵀ Qᵢ) ⊙ (Wᵀ Kⱼ))
  = U · ((Vᵀ Qᵢ) ⊙ Σⱼ (Wᵀ Kⱼ))
  = U · ((Vᵀ Qᵢ) ⊙ cumsum_j(Wᵀ Kⱼ)ᵢ)
```

The $(V^T Q_i)$ is constant in j and factors out of the sum. The j cumulative sum of $W^T K_j$ becomes a (B, L, r) prefix-sum along the sequence axis — a single pass, no pairwise materialization. The complexity drops from $O(B \cdot L^2 \cdot D \cdot K)$ to $O(B \cdot L \cdot r \cdot K)$.

3.2 Factorizability requires query-side routing

Extending to MoBE:

```
Σⱼ wₖ(Qᵢ, Kⱼ) · BCNₖ(Qᵢ, Kⱼ)
```

If the routing weight $w_k(Q_i, K_j)$ depends on both Q_i and K_j, it does **not** factor out of the sum over j, and the $O(L^2)$ blowup returns. If the routing depends only on Q_i, it factors:

```
Σⱼ wₖ(Qᵢ) · BCNₖ(Qᵢ, Kⱼ)
  = wₖ(Qᵢ) · Σⱼ BCNₖ(Qᵢ, Kⱼ)
  = wₖ(Qᵢ) · Uₖ · ((Vₖᵀ Qᵢ) ⊙ cumsum_j(Wₖᵀ Kⱼ)ᵢ)
```

We therefore adopt **query-side routing**: $w_k(Q_i) = \mathrm{softmax}(\mathrm{Router}(Q_i))$. This aligns with standard token-level MoE routing in modern LLMs (Shazeer et al.; Fedus et al.; Jiang et al.) — the router chooses experts per query token, not per $(query, key)$ pair. Expert specialization survives in the per-token form; the algebraic results of Dovzak 2026a are compatible with this restriction (an ablation on Power 2022 is reported in Section 6).

3.3 Vectorized K-expert kernel

The naive MoBE implementation has a Python loop over experts:

```
outs = torch.stack([eₖ(x, y) for eₖ in self.experts], dim=-1)
```

This serializes K kernel launches and prevents GPU pipelining. We replace it with stacked parameters and batched einsum:

```
V_all = torch.stack([Vₖ for Vₖ in expertsV], dim=0)   # (K, D, r)
xV  = torch.einsum('bld,kdr->blkr', x, V_all)          # (B, L, K, r)
yW  = torch.einsum('bld,kdr->blkr', y, W_all)          # (B, L, K, r)
prod_kr = (xV * yW).reshape(B, L, K*r)                 # (B, L, K·r)
out = prod_kr @ U_merged                               # (B, L, D)
# merged U: (K*r, D) flattened from (K, D, r)
```

A single batched matmul replaces K sequential kernel launches. Functionally identical (verified numerically, max $|\Delta| < 1e\text{-}7$ vs naive CPU reference); throughput improves by ~3× in isolation.

3.4 Combined: full kernel

Putting 3.1 + 3.2 + 3.3 together, the causal MoBE-BCN forward becomes:

```
def forward(self, x):
    Q = self.W_Q(x); K = self.W_K(x)
    # Project per-expert: (B, L, K, r)
    xV = einsum('bld,kdr->blkr', Q, V_all)
    yW = einsum('bld,kdr->blkr', K, W_all)
    # Causal cumsum over j
    cum_yW = yW.cumsum(dim=1)                          # (B, L, K, r)
    prod   = xV * cum_yW                               # (B, L, K, r)
    # Q-side router
    w = softmax(self.router(Q), dim=-1)                # (B, L, K)
    prod_w = prod * w.unsqueeze(-1)                    # (B, L, K, r)
    # Project to D via stacked U
    out = einsum('blkr,kdr->bld', prod_w, U_all)       # (B, L, D)
    # Normalize by |valid j ≤ i|
    n_valid = arange(1, L+1, device=x.device).view(1, L, 1)
    return self.W_O(out / n_valid)
```

Correctness verified against a naive pairwise+routing reference implementation (max absolute error < 1e-7 on CPU; < 1e-5 under bf16).

3.5 Benchmark

On a single RTX 4090 at model config `d=512, L=256, n_layers=12, K=4, rank=128` (98M params total, bf16 autocast, no gradient checkpointing):

Implementation	tok/s	VRAM peak	24h token budget
Naive Python loop + pairwise expand	288	15 GB	25M
+ vectorized K experts	853	8 GB	74M
+ cumsum factorization	6,006	1.3 GB	519M
+ batch size 64 (memory now allows)	**74,654**	19 GB	**6.5B**

259× end-to-end speedup. The `O(L)` memory frees the batch-size budget, which is what unlocks the final order-of-magnitude gain.

---

4. Kazdov-α Training Setup

`[PARTIAL — final numbers pending the complete training run.]`

- **Model:** KazdovLM 98M parameters — `d=512, L=256, n_layers=12, n_heads=8, rank=128, K=4, mlp_dim=2048, max_len=256`. Hybrid block: cumsum-factorized MoBE-BCN + standard causal MHA in parallel.
- **Tokenizer:** GPT-2 BPE (50,257 vocab).
- **Data (5.95B tokens):**

- OpenWebMath 45% (2.45B tokens)
- proof-pile v1 22% (1.40B)
- MathPile-Commercial 15% (1.05B)
- AutoMathText top-quality slice 10% (0.70B)
- NuminaMath-CoT 8% (0.35B)

- **Optimization:** AdamW lr=5e-4, betas=(0.9, 0.95), weight decay 0.1, grad clip 1.0, WSD scheduler (warmup 500 steps, stable, linear decay in last 20%).
- **Hardware:** single RTX 4090, bf16 autocast, batch 32, length 256 → ~60-70K tok/s sustained.
- **Target:** 5B tokens = ~610k steps ≈ 23h wall time.
- **Held-out:** last 0.5% of each data shard, deterministic eval.

---

5. Results (pending)

This section will be populated once training completes.

5.1 Held-out math LM loss

Model	Params	Tokenizer	open-web-math	proof-pile v1	MathPile	AutoMathText	NuminaMath	MEA
GPT-2 124M (baseline)	124M	GPT-2	3.57	3.12	3.04	3.28	2.58	**3.12**
Kazdov-α	98M	GPT-2	`[TBD]`	`[TBD]`	`[TBD]`	`[TBD]`	`[TBD]`	`[TBD`

5.2 Loss curves

`[figure pending]` — training-loss and held-out-loss curves vs. tokens seen.

5.3 Routing diagnostics

`[figure pending]` — per-layer expert usage entropy over training, expert-specialization heatmap.

---

6. Ablations (pending)

Planned ablation sweep (~200M tokens each):

Ablation	Config	Held-out loss	Δ vs base
Base (`K=4, rank=128, hybrid-MHA`)	—	`[TBD]`	0
K = 2	half experts	`[TBD]`	`[TBD]`
K = 8	double experts	`[TBD]`	`[TBD]`
rank = 64	narrower bilinear	`[TBD]`	`[TBD]`
rank = 256	wider bilinear	`[TBD]`	`[TBD]`
No hybrid MHA	pure MoBE attention	`[TBD]`	`[TBD]`
Q+K routing (naive $O(L^2)$)	confirms Q-only equivalent on short L	`[TBD]`	`[TBD]`

---

7. Qualitative samples (pending)

Prompt–generation pairs from the final checkpoint — LaTeX completions, derivative/integral rules, short problem sketches.

---

8. Discussion (draft)

Positioning. Kazdov-α is a demonstration of the cumsum-factorized MoBE-BCN kernel at LM scale, not a production language model. At 98M parameters and 5B tokens it is sub-Chinchilla-optimal by about a factor of 10 on breadth; its purpose is architectural validation, not capability-claims. Downstream benchmarks (GSM8K, MATH) are not meaningfully evaluable at this parameter count.

Relation to linear attention. The cumsum trick is the standard linearization technique for any attention whose kernel is separable across `Q` and `K` (Katharopoulos et al., 2020). Our contribution is identifying that (a) bilinear composition attention is exactly such a kernel, and (b) MoBE-BCN preserves separability precisely when routing is query-side, which is the MoE convention anyway. The combination yields an `O(L)` bilinear-composition attention with learned expert specialization.

Limitations.

- Short context (256 tokens in the alpha; scaling to 2048 is straightforward with the `O(L)` kernel but not evaluated here).
- Single-author, single-GPU training — reproducibility relies on the open release rather than independent confirmation.
- No SFT or instruction-tuning; model is base-LM only.
- GSM8K / MATH benchmarks not meaningful at 98M; a 1B-scale follow-up is the correct target for capability claims.

References (draft)

- Choromanski, K. et al. (2020). *Rethinking Attention with Performers.* arXiv:2009.14794.
- Dovzak, J. C. (2026a). *MoBE-BCN: Mixture of Bilinear Experts for Sample-Efficient Algebraic Composition Learning.* (This repository, `papers/01-mobe-bcn.md`.)
- Eldan, R., Li, Y. (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv:2305.07759.
- Fedus, W., Zoph, B., Shazeer, N. (2022). *Switch Transformers.* arXiv:2101.03961.
- Jiang, A. et al. (2024). *Mixtral of Experts.* arXiv:2401.04088.
- Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F. (2020). *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.* ICML 2020.
- Paster, K., Dos Santos, M., Azerbayev, Z., Ba, J. (2023). *OpenWebMath.* arXiv:2310.06786.
- Shazeer, N. et al. (2017). *Outrageously Large Neural Networks.* arXiv:1701.06538.
- Toshniwal, S. et al. (2024). *OpenMathInstruct / NuminaMath-CoT.* [HuggingFace collections].

How to cite (once complete)

```
@article{dovzak2026kazdov,
  title   = {Kazdov-LM: Scaling MoBE-BCN to Math Language Modeling
             via Cumsum Factorization},
  author  = {Dovzak, Juan Cruz},
  year    = {2026},
  note    = {In preparation. Companion to Dovzak 2026a.
             https://github.com/OriginalKazdov/kazdov},
}
```